

# Hardware and Software Foundations of Intelligent Computing Systems

## Syllabus

### Course Information

**Course Number:** 04832240

**Credits/Hours:** 3 Credits / 48 Hours (32 Lecture Hours + 16 Lab Hours)

**Semester:** Fall 2026

**Class Schedule:** (See course registration system)

**Classroom:** (See course registration system)

**Department:** School of Computer Science, Peking University

**Target Programs:** Computer Science and Technology, Software Engineering, Artificial Intelligence

**Course Type:** Elective (Computer Architecture and Parallel Computing Track)

### Instructor

**Lead Instructor:** Luo Guojie

**Email:** [gluo@pku.edu.cn](mailto:gluo@pku.edu.cn)

**Office Hours:** (TBD; appointments available via email)

### Course Description

This course focuses on the fundamental requirements for high-performance computing system software in the era of artificial intelligence, providing a systematic treatment of key technologies spanning from hardware architecture to the full software stack. Topics include heterogeneous computing programming models, deep learning framework internals, AI operator development and optimization, AI compiler principles, and distributed parallel training.

The course balances theoretical instruction with engineering practice, helping students build a comprehensive understanding of the full stack from hardware to system software, frameworks, and applications, thereby equipping them with foundational capabilities for intelligent computing systems R&D.

# Course Overview

| Week | Module                                                         | Main Content                                                                                           |
|------|----------------------------------------------------------------|--------------------------------------------------------------------------------------------------------|
| 1    | Module 1: AI System Fundamentals & Heterogeneous Computing     | Overview of AI hardware architecture; challenges to Moore's Law and trends in heterogeneous computing  |
| 2    |                                                                | SIMT programming model; thread hierarchy and memory hierarchy                                          |
| 3    |                                                                | Analysis of deep learning framework call chains; Shared Memory and Bank Conflict                       |
| 4    |                                                                | Cache locality optimization; Tiled matrix multiplication; Introduction to Triton                       |
| 5    | Module 2: High-Performance AI Operators & Operator Engineering | Principles of Transformer attention mechanism; design and implementation of FlashAttention             |
| 6    |                                                                | Optimization of convolution, normalization, and activation function operators                          |
| 7    |                                                                | Operator fusion techniques; debugging and profiling of complex operators                               |
| 8    |                                                                | Q&A and discussion on Module 1 / Module 2 labs                                                         |
| 9    | Module 3: AI Compiler Principles & Optimization                | Overview of AI compilers; fundamentals of Intermediate Representation (IR); SSA and computation graphs |
| 10   |                                                                | In-depth analysis of MLIR architecture                                                                 |
| 11   |                                                                | Operator fusion passes: graph matching, pattern recognition, and subgraph rewriting                    |
| 12   |                                                                | Layout and memory planning; multi-chip backend code generation                                         |
| 13   | Module 4: Distributed Parallel Training & Communication        | Principles of data parallelism, tensor parallelism, and pipeline parallelism                           |
| 14   |                                                                | Communication primitives and topology optimization; hybrid parallelism and memory optimization         |
| 15   | Module 5: Performance Evaluation & Frontier Outlook            | Operator performance metrics; automatic operator tuning; LLM-assisted code generation                  |

## Course Platform and Online Laboratory Environment

### FlagOS Intelligent Computing System Software Stack

The laboratory environment for this course is built on the FlagOS Intelligent Computing System Software Stack. Developed by the Beijing Academy of Artificial Intelligence (BAAI), FlagOS is a unified open-source heterogeneous computing software stack designed for diverse computing architectures, enabling a "develop once, run on multiple chips, deploy everywhere" paradigm. FlagOS comprises several core components, including:

- **FlagScale**: A distributed training/inference framework supporting large model training and deployment across heterogeneous hardware
- **FlagGems**: A general-purpose operator library consisting of high-performance AI operators implemented in the Triton language
- **FlagTree**: A unified compiler supporting Triton compilation for multiple AI chip backends
- **FlagCX**: A cross-chip communication library supporting high-performance collective communication in heterogeneous computing environments

All laboratory tasks, including Triton kernel development, operator optimization, MLIR pass implementation, and distributed training deployment, will be carried out within the FlagOS ecosystem.

### Online Laboratory

The course provides a cloud-based online laboratory platform. Students can remotely access GPU computing resources through a web browser to complete all course experiments without configuring a local GPU environment.

#### Platform Registration and Login:

- After the semester begins, the instructor will set up online laboratory accounts for all enrolled students. Please access the platform at the following URL:

Online Laboratory Portal: <https://flagos.io/OnlineLab>

- Once your account is activated, you will be allocated GPU computing resources

and can write, run, and debug code directly online. Please use computing time responsibly and avoid occupying resources for extended periods without releasing them.

### Platform Usage Guide:

- For detailed documentation on the online laboratory platform (environment setup, experiment submission procedures, FAQs, etc.), please refer to:

Platform Documentation:

<https://docs.flagos.io/projects/onlinelaboratory/en/latest/onlinelaboratory.html>

- Students are advised to carefully read the above documentation before the first experiment to familiarize themselves with the basic workflow. If you encounter technical issues, please consult the documentation first, then seek help through the course discussion forum or Office Hours.

### Important Notes:

- The computing resources on the online laboratory platform are exclusively for course experiments. Do not use them for personal projects unrelated to the course.
- Please save your code and data locally in a timely manner after completing experiments, as platform instances may be periodically reclaimed.
- If resources are unavailable during peak hours, please schedule your work during off-peak times and plan your experiment schedule accordingly.

## Course-Related Links

Below are important links you may need during the course:

| Resource                   | URL                                                                                                                                                                             |
|----------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| FlagOS Official Website    | <a href="https://flagos.io">https://flagos.io</a>                                                                                                                               |
| Online Laboratory          | <a href="https://flagos.io/OnlineLab">https://flagos.io/OnlineLab</a>                                                                                                           |
| Platform Documentation     | <a href="https://docs.flagos.io/projects/onlinelaboratory/en/latest/onlinelaboratory.html">https://docs.flagos.io/projects/onlinelaboratory/en/latest/onlinelaboratory.html</a> |
| FlagOS Developer Community | <a href="https://flagos.csdn.net">https://flagos.csdn.net</a>                                                                                                                   |

|                    |                                                                                           |
|--------------------|-------------------------------------------------------------------------------------------|
| y                  |                                                                                           |
| FlagGems<br>GitHub | <a href="https://github.com/FlagOpen/FlagGems">https://github.com/FlagOpen/FlagGems</a>   |
| FlagTree<br>GitHub | <a href="https://github.com/flagos-ai/FlagTree">https://github.com/flagos-ai/FlagTree</a> |
| FlagCX<br>GitHub   | <a href="https://github.com/flagos-ai/FlagCX">https://github.com/flagos-ai/FlagCX</a>     |

## Prerequisites

Before enrolling in this course, you should have the following background:

- Computer Organization: Understanding of basic processor architecture, memory hierarchy, and instruction execution pipeline.
- Operating Systems: Familiarity with fundamental concepts such as process/thread management and memory management.
- Introduction to Deep Learning: Familiarity with basic neural network concepts and hands-on experience with PyTorch or TensorFlow.
- C/C++ Programming: Solid proficiency in C/C++ programming.

If you have no prior experience with CUDA, Triton, or GPU programming, there is no need to worry—the course will start from the fundamentals. However, the course progresses at a brisk pace, so it is recommended that you familiarize yourself with basic Python and PyTorch usage in advance.

## Course Objectives

Upon completing this course, you will be able to:

1. Gain an in-depth understanding of the architectural characteristics of heterogeneous AI hardware (CPU, GPU, NPU), master the SIMT programming model, proficiently develop and optimize fundamental AI operators using CUDA/Triton, and analyze performance bottlenecks using the Roofline model.
2. Understand the complete execution pipeline of mainstream deep learning frameworks from the Python frontend to hardware execution, master the principles of automatic differentiation and computation graph optimization, and develop custom operators integrated into the framework.
3. Understand the fundamental principles of AI compilers (TVM, XLA,

TorchDynamo, etc.), master the basic concepts of intermediate representation (IR), and gain familiarity with common compiler optimizations such as operator fusion and memory planning.

4. Understand the core paradigms of large-scale distributed training (data parallelism, model parallelism, pipeline parallelism), master mainstream communication primitives, and acquire the ability to deploy distributed training tasks in multi-GPU/multi-node environments.

## Textbooks and References

There is no required textbook for this course. Reading materials primarily consist of lecture notes, research papers, and online documentation. The following are recommended reference resources:

### Recommended Reference Books

- Zheng Weimin, Zhang Chenxi, et al. Intelligent Computing Systems. Tsinghua University Press, 2020.
- NVIDIA. 《CUDA C++ Best Practices Guide》. NVIDIA Developer Documentation, 2024.

### Key Papers

- Philippe Tillet, et al. Triton: An Intermediate Language and Compiler for Tiled Neural Network Computations. MAPL, 2019.
- Tri Dao, et al. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. NeurIPS, 2022.
- Tianqi Chen, et al. TVM: An Automated End-to-End Optimizing Compiler for Deep Learning. OSDI, 2018.
- Chris Lattner, et al. MLIR: Scaling Compiler Infrastructure for Domain Specific Computation. CGO, 2021.
- Shoeybi, et al. Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism. arXiv, 2019.

Additional reading materials and lecture notes will be distributed during the course. Please follow the course homepage and announcements.

## Laboratory Assignments

The course includes 4 module-based laboratory assignments, each corresponding to one of the four core modules. Each assignment contains both basic tasks and

advanced tasks—basic tasks verify your understanding of core principles, while advanced tasks encourage exploratory optimization. All experiments are to be completed on the FlagOS online laboratory platform.

A brief description of each experiment is as follows:

- **Lab 1 · Softmax Kernel:** Implement a Softmax kernel using Triton. The code must pass numerical accuracy tests, and an experiment report describing the implementation approach must be submitted.
- **Lab 2 · Operator Fusion:** Implement a fused RMSNorm + Residual Connection operator. Performance must reach at least 90% of the baseline implementation. Submit the code along with a performance comparison report.
- **Lab 3 · MLIR Pass Implementation:** Write a simple MLIR optimization pass to implement Conv+BN fusion. The pass must pass all test cases. Submit the code along with a description of the optimization results.
- **Lab 4 · Distributed Training:** Deploy distributed training of a 7B-parameter model in a multi-GPU environment. Record throughput, linear scaling efficiency, and GPU memory utilization data.

Please plan your time wisely and start experiments early—debugging and optimization of programming tasks often take longer than expected. If you encounter difficulties, make good use of Office Hours and the course discussion forum.

## Course Project

A comprehensive course project is assigned at the end of the semester (individual or team-based; teams of 2–3 members are recommended). You are required to conduct performance analysis and optimization on a specific AI operator or system software module, and present your design approach, experimental data, and performance improvement results in a defense presentation.

The project is divided into two phases:

- **Design Proposal (Midterm):** Submit a topic selection report presenting the project background, algorithmic approach, and preliminary experimental progress.
- **Final Deliverables (End of Term):** Submit a complete code repository and technical report. Present the final implementation results and performance benchmarking data in a defense presentation.

## Examination

The course includes one final examination covering all course content. The exam consists of multiple-choice questions, fill-in-the-blank questions, and short-answer/analytical questions, with an emphasis on assessing your understanding of

core concepts and your ability to apply the knowledge learned to solve practical engineering problems.

The exam is a closed-book written test. The specific date and location will be announced separately.

## Grading Policy

Your final grade is composed of the following three weighted components:

| Assessment Component | Weight | Description                                                                                                                                                               |
|----------------------|--------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Lab Assignments (4)  | 40%    | Each of the four module experiments accounts for 10%, graded on code correctness, performance, and experiment report quality                                              |
| Course Project       | 30%    | Design proposal accounts for 9%, final deliverables for 21%; grading dimensions include performance improvement, implementation quality, report, and defense presentation |
| Final Examination    | 30%    | Multiple-choice: 30 pts + Fill-in-the-blank: 20 pts + Short-answer/Analytical: 50 pts                                                                                     |

## Getting Help

If you encounter problems during your studies, there are several ways to get help:

- **Office Hours:** The instructor and teaching assistants hold Q&A sessions during designated time slots; see the course homepage for the specific schedule. For more in-depth questions, it is recommended to schedule an appointment via email in advance.
- **Course Discussion Forum:** You are encouraged to ask questions and engage in discussions on the course forum or WeChat group. Before reaching out to the instructor, try discussing with your classmates first—peer discussions can often be highly insightful.
- **Online Laboratory Issues:** If you encounter technical issues with the platform, please first consult the [Platform Documentation](#), then provide feedback through the course discussion forum or Office Hours.



# Collaboration and Academic Integrity

We encourage communication and collaboration among students—forming study groups and discussing problem-solving approaches are excellent learning methods. However, please observe the following principles:

- You may freely discuss ideas and approaches for experiments and assignments, but each student must independently complete and submit their own code and report.
- If your problem-solving approach benefited from discussions with others or public resources, please include an acknowledgment in your report.
- Plagiarism of others' code or reports is prohibited, as is submitting solutions directly generated by AI tools without proper understanding and modification.
- For team-based course projects, the report must clearly specify each member's division of labor and contributions.